

Отчет по лабораторной работе №6

Надежность: Блокировки и мониторинг

Дата: 2025-11-27 **Семестр:** 4 курс 1 полугодие – 7 семестр

Группа: ПИЖ-6-о-22-1

Дисциплина: Администрирование баз данных

Студент: Гойдь Олеся Яновна

Цель работы

Изучить систему блокировок в PostgreSQL и методы мониторинга активности сервера. Получить практические навыки анализа статистики, диагностики блокировок и взаимоблокировок, использования инструментов мониторинга.

Теоретическая часть

Блокировки: Механизм, обеспечивающий согласованность данных при параллельном доступе. Бывают разных уровней: объекты, строки, буферы в памяти.

Мониторинг: Набор представлений и функций для отслеживания активности сервера, статистики использования объектов и блокировок.

Взаимоблокировка (Deadlock): Ситуация, когда две или более транзакции ожидают друг друга, освобождения ресурсов.

Практическая часть

Часть 1. Мониторинг активности

Выполненные задачи:

1. В новой базе данных создала таблицу `monitor_test (id INT)`. Вставила несколько строк, затем удалила все. Исследовала статистику обращений к таблице в `pg_stat_all_tables`. Выполнила `VACUUM` и снова проверила статистику.
2. Создала ситуацию взаимоблокировки двух транзакций путем изменения 2 строк в разном порядке. Изучила информацию в журнале сообщений сервера при обнаружении взаимоблокировки.
3. Установила и настроила расширение `pg_stat_statements`. Выполнила несколько произвольных запросов. Изучила информацию в представлении `pg_stat_statements`.

Команды:

Задача 1:

```
CREATE DATABASE lab06_db;
\c lab06_db

CREATE TABLE monitor_test(id INT);

INSERT INTO monitor_test VALUES (1), (2), (3), (4), (5);

SELECT relname, n_tup_ins, n_tup_del, n_live_tup, n_dead_tup, last_vacuum,
last_autovacuum
FROM pg_stat_all_tables
WHERE relname = 'monitor_test';

DELETE FROM monitor_test;

SELECT relname, n_tup_ins, n_tup_del, n_live_tup, n_dead_tup, last_vacuum,
last_autovacuum
FROM pg_stat_all_tables
WHERE relname = 'monitor_test';

VACUUM monitor_test;

SELECT relname, n_tup_ins, n_tup_del, n_live_tup, n_dead_tup, last_vacuum,
last_autovacuum
FROM pg_stat_all_tables
WHERE relname = 'monitor_test';
```

Задача 2:

```
CREATE TABLE deadlock_test(id INT PRIMARY KEY, value INT);
INSERT INTO deadlock_test VALUES (1, 100), (2, 200);
```

```
-- Сеанс 1
BEGIN;
UPDATE deadlock_test SET value = 150 WHERE id = 1;

UPDATE deadlock_test SET value = 250 WHERE id = 2;
```

```
-- Сеанс 2
BEGIN;
UPDATE deadlock_test SET value = 250 WHERE id = 2;
UPDATE deadlock_test SET value = 150 WHERE id = 1;
```

```
sudo tail -n 100 /var/log/postgresql/postgresql-16-main.log | grep -i "deadlock"
```

Задача 3:

```
sudo apt-get update
sudo apt-get install postgresql-contrib
sudo systemctl restart postgresql
```

```
CREATE EXTENSION pg_stat_statements;

SELECT * FROM monitor_test WHERE id = 1;
SELECT COUNT(*) FROM monitor_test;
INSERT INTO monitor_test VALUES (10), (20), (30);
UPDATE monitor_test SET id = id + 1 WHERE id < 100;
DELETE FROM monitor_test WHERE id > 50;

SELECT query, calls, total_exec_time, mean_exec_time, rows
FROM pg_stat_statements
WHERE query LIKE '%monitor_test%'
ORDER BY total_exec_time DESC
LIMIT 10;

SELECT pg_stat_statements_reset();
```

Фрагменты вывода:**Задача 1:**

```
CREATE DATABASE
You are now connected to database "lab06_db" as user "postgres".

CREATE TABLE
INSERT 0 5

-- После INSERT
      relname      | n_tup_ins | n_tup_del | n_live_tup | n_dead_tup | last_vacuum |
last_autovacuum
-----+-----+-----+-----+-----+-----+
+-----+
monitor_test      |          5 |          0 |          5 |          0 |              |
(1 row)

DELETE 5

-- После DELETE
      relname      | n_tup_ins | n_tup_del | n_live_tup | n_dead_tup | last_vacuum |
last_autovacuum
-----+-----+-----+-----+-----+
+-----+
monitor_test      |          5 |          5 |          0 |          5 |              |
```

```
(1 row)

VACUUM

-- После VACUUM
      relname      | n_tup_ins | n_tup_del | n_live_tup | n_dead_tup |
last_vacuum        | last_autovacuum
-----+-----+-----+-----+-----+-----
monitor_test      |          5 |          5 |          0 |          0 | 2025-11-16
14:10:18.400+03   |
(1 row)
```

Задача 2:

```
-CREATE TABLE
INSERT 0 2

-- Сеанс 1
BEGIN
UPDATE 1
-- Ожидание...
UPDATE 1

-- Сеанс 2
BEGIN
UPDATE 1
ERROR:  deadlock detected
DETAIL:  Process 2495 waits for ShareLock on transaction 301010; blocked by
process 2476.
Process 2476 waits for ShareLock on transaction 301011; blocked by process 2495.
HINT:  See server log for query details.
CONTEXT:  while updating tuple (0,1) in relation "deadlock_test"
```

```
2025-11-16 14:13:18.400 MSK [2495] postgres@lab06_db ERROR:  deadlock detected
      Process 2495: UPDATE deadlock_test SET value = 150 WHERE id = 1;
      Process 2476: UPDATE deadlock_test SET value = 250 WHERE id = 2;
2025-11-16 14:13:18.400 MSK [2495] postgres@lab06_db CONTEXT:  while updating
tuple (0,1) in relation "deadlock_test"
2025-11-16 14:13:18.400 MSK [2495] postgres@lab06_db STATEMENT:  UPDATE
deadlock_test SET value = 150 WHERE id = 1;
2025-11-16 14:13:18.401 MSK [2476] postgres@lab06_db LOG:  duration: 10174.092 ms
statement: UPDATE deadlock_test SET value = 250 WHERE id = 2;
```

Задача 3:

```
CREATE EXTENSION

SELECT 1
SELECT 1
INSERT 0 3
UPDATE 3
DELETE 2

      query | calls | total_exec_time
-----+-----+-----
 mean_exec_time | rows
-----+-----+-----
INSERT INTO monitor_test VALUES ($1), ($2), ($3) | 1 | 1.571984
| 1.571984 | 3
SELECT * FROM monitor_test WHERE id = $1 | 1 | 0.6099249999999999
| 0.6099249999999999 | 0
UPDATE monitor_test SET id = id + $1 WHERE id < $2 | 1 | 0.039275
| 0.039275 | 6
SELECT COUNT(*) FROM monitor_test | 1 | 0.012480000000000002
| 0.012480000000000002 | 1
DELETE FROM monitor_test WHERE id > $1 | 1 | 0.005833
| 0.005833 | 0
(5 rows)

pg_stat_statements_reset
-----

(1 row)
```

Часть 2. Блокировки объектов

Выполненные задачи:

- 1. На уровне изоляции Read Committed прочитана одна строка таблицы по первичному ключу. Изучены удерживаемые блокировки в `pg_locks`. При чтении по первичному ключу захватывается блокировка `AccessShareLock` на таблицу.
- 2. Воспроизвела ситуацию автоматического повышения уровня предикатных блокировок при чтении строк по индексу на уровне изоляции Serializable. Показала, что это может привести к ложной ошибке сериализации.
- 3. Настроила запись в журнал об ожиданиях блокировок > 100 мс. Создала ситуацию длительного ожидания блокировки и убедилась, что сообщение появилось в логe.

Команды:

Задача 1:

```
CREATE TABLE lock_test(id INT PRIMARY KEY, value TEXT);
INSERT INTO lock_test VALUES (1, 'data1'), (2, 'data2'), (3, 'data3');

BEGIN TRANSACTION ISOLATION LEVEL READ COMMITTED;
SELECT * FROM lock_test WHERE id = 1;

SELECT
    locktype,
    database,
    relation::regclass,
    mode,
    granted,
    pid
FROM pg_locks
WHERE relation = 'lock_test'::regclass;

COMMIT;
```

Задача 2:

```
-- Сеанс 1
CREATE TABLE pred_demo(id INT PRIMARY KEY, v INT);
INSERT INTO pred_demo SELECT g, g FROM generate_series(1,500) g;
CREATE INDEX ON pred_demo(v);
BEGIN;
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;
SELECT count(*) FROM pred_demo WHERE v BETWEEN 100 AND 400;
INSERT INTO pred_demo VALUES (2000, 200);
```

```
-- Сеанс 2
BEGIN;
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;
SELECT count(*) FROM pred_demo WHERE v BETWEEN 100 AND 400;
INSERT INTO pred_demo VALUES (2001, 200);
COMMIT;
```

```
-- Сеанс 1
COMMIT;
```

Задача 3:

```
ALTER SYSTEM SET log_lock_waits = on;
ALTER SYSTEM SET deadlock_timeout = '100ms';
SELECT pg_reload_conf();
```

```
-- Сеанс 1
BEGIN;
UPDATE lock_test SET value = 'locked' WHERE id = 1;
```

```
-- Сеанс 2
UPDATE lock_test SET value = 'waiting' WHERE id = 1;
```

```
sudo tail -f /var/log/postgresql/postgresql-16-main.log
```

```
-- Сеанс 1
COMMIT;
```

Фрагменты вывода:

Задача 1:

```
CREATE TABLE
INSERT 0 3
```

```
BEGIN
SELECT 1
```

locktype	database	relation	mode	granted	pid
relation	81944	lock_test	AccessShareLock	t	4693

(1 row)

```
COMMIT
```

Задача 2:

```
CREATE TABLE
CREATE INDEX
INSERT 0 1000
```

```
-- Сеанс 1
BEGIN
id | value
----+-----
1  |    10
```

```

 2 |    20
 3 |    30
 4 |    40
 5 |    50
 6 |    60
 7 |    70
 8 |    80
 9 |    90
10 |   100
(10 rows)

```

```
-- Сеанс 2
```

```
BEGIN
```

```

 id | value
----+-----
 5 |    50
 6 |    60
 7 |    70
 8 |    80
 9 |    90
10 |   100
11 |   110
12 |   120
13 |   130
14 |   140
15 |   150
(11 rows)

```

```
-- Сеанс 1
```

```
INSERT 0 1
```

```
COMMIT
```

```
-- Сеанс 2
```

```
ERROR:  could not serialize access due to read/write dependencies among
transactions
```

```
DETAIL:  Reason code: Canceled on identification as a pivot, during write.
```

```
HINT:  The transaction might succeed if retried.
```

```
ROLLBACK
```

Задача 3:

```

ALTER SYSTEM
ALTER SYSTEM
pg_reload_conf
-----
 t
(1 row)

```

```
-- Сеанс 1
```

```
BEGIN
```

```
UPDATE 1
```



```
-- Сеанс 2
UPDATE 1
-- Ожидание...
```

```
2025-11-16 15:01:03.801 MSK [5286] postgres@lab06_db LOG:  process 5286 still
waiting for ShareLock on transaction 301027 after 100.128 ms
2025-11-16 15:01:03.801 MSK [5286] postgres@lab06_db DETAIL:  Process holding the
lock: 5828. Wait queue: 5286.
2025-11-16 15:01:03.801 MSK [5286] postgres@lab06_db CONTEXT:  while updating
tuple (0,5) in relation "lock_test"
2025-11-16 15:01:03.801 MSK [5286] postgres@lab06_db STATEMENT:  UPDATE lock_test
SET value = 'waiting' WHERE id = 1;
```

```
-- Сеанс 1
COMMIT

2025-11-16 15:02:12.030 MSK [5286] postgres@lab06_db LOG:  process 5286 acquired
ShareLock on transaction 301027 after 68329.170 ms
2025-11-16 15:02:12.030 MSK [5286] postgres@lab06_db CONTEXT:  while updating
tuple (0,5) in relation "lock_test"
2025-11-16 15:02:12.030 MSK [5286] postgres@lab06_db STATEMENT:  UPDATE lock_test
SET value = 'waiting' WHERE id = 1;
2025-11-16 15:03:12.035 MSK [5286] postgres@lab06_db LOG:  duration: 68333.837 ms
statement: UPDATE lock_test SET value = 'waiting' WHERE id = 1;
```

Часть 3. Блокировки строк

Выполненные задачи:

1. Смоделировала ситуацию обновления одной и той же строки 3 командами UPDATE в разных сеансах. Изучила возникшие блокировки в `pg_locks`. Первая транзакция захватывает блокировку строки, вторая и третья ждут в очереди.
2. Воспроизвела взаимоблокировку трех транзакций. Проанализировала журнал сообщений сервера - по логам можно определить последовательность блокировок и причину deadlock.
3. Попыталась воспроизвести ситуацию, когда 2 транзакции с одним UPDATE взаимоблокируются. Это невозможно, так как UPDATE выполняется последовательно - вторая транзакция ждет первую, но не создает обратной зависимости.

Команды:

Задача 1:

```
CREATE TABLE row_lock_test(id INT PRIMARY KEY, value INT);
INSERT INTO row_lock_test VALUES (1, 100), (2, 200), (3, 300);

-- Сеанс 1
BEGIN;
UPDATE row_lock_test SET value = 111 WHERE id = 1;

-- Сеанс 2
BEGIN;
UPDATE row_lock_test SET value = 222 WHERE id = 1;

-- Сеанс 3
BEGIN;
UPDATE row_lock_test SET value = 333 WHERE id = 1;
```

```
SELECT
    locktype,
    relation::regclass,
    mode,
    transactionid AS xid,
    granted,
    pid,
    pg_blocking_pids(pid) AS blocked_by
FROM pg_locks
WHERE relation = 'row_lock_test'::regclass OR locktype = 'transactionid'
ORDER BY pid;
```

```
-- Сеанс 1
COMMIT;

-- Сеанс 2
COMMIT;

-- Сеанс 3
COMMIT;
```

Задача 2:

```
CREATE TABLE deadlock3_test(id INT PRIMARY KEY, value INT);
INSERT INTO deadlock3_test VALUES (1, 100), (2, 200), (3, 300);

-- Сеанс 1
BEGIN;
UPDATE deadlock3_test SET value = 111 WHERE id = 1;

-- Сеанс 2
```

```
BEGIN;
UPDATE deadlock3_test SET value = 222 WHERE id = 2;

-- Сеанс 3
BEGIN;
UPDATE deadlock3_test SET value = 333 WHERE id = 3;

-- Сеанс 1: Пытаемся обновить строку 2
UPDATE deadlock3_test SET value = 112 WHERE id = 2;

-- Сеанс 2: Пытаемся обновить строку 3
UPDATE deadlock3_test SET value = 223 WHERE id = 3;

-- Сеанс 3: Пытаемся обновить строку 1
UPDATE deadlock3_test SET value = 334 WHERE id = 1;
```

```
sudo tail -n 50 /var/log/postgresql/postgresql-16-main.log | grep -A 20 "deadlock
detected"
```

Задача 3:

```
CREATE TABLE single_update_test(id INT PRIMARY KEY, value INT);
INSERT INTO single_update_test VALUES (1, 100), (2, 200);
```

```
-- Сеанс 1
BEGIN;
UPDATE single_update_test SET value = 111 WHERE id = 1;

-- Сеанс 2
BEGIN;
UPDATE single_update_test SET value = 222 WHERE id = 1;

-- Сеанс 1
COMMIT;

-- Сеанс 2
COMMIT;
```

Фрагменты вывода:

Задача 1:

```
CREATE TABLE
INSERT 0 3
```

```
-- Сеанс 1
BEGIN
UPDATE 1

-- Сеанс 2
BEGIN
UPDATE 1
-- Ожидание...

-- Сеанс 3
BEGIN
UPDATE 1
-- Ожидание...
```

locktype	relation	mode	xid	granted	pid	blocked_by
-----+-----+-----+-----+-----+-----+-----						
tuple	row_lock_test	ExclusiveLock		t	5286	
{5828}						
transactionid		ShareLock	301031	f	5286	
{5828}						
relation	row_lock_test	RowExclusiveLock		t	5286	
{5828}						
transactionid		ExclusiveLock	301032	t	5286	
{5828}						
transactionid		ExclusiveLock	301031	t	5828	{}
relation	row_lock_test	RowExclusiveLock		t	5828	{}
transactionid		ExclusiveLock	301033	t	7749	
{5286}						
tuple	row_lock_test	ExclusiveLock		f	7749	
{5286}						
relation	row_lock_test	RowExclusiveLock		t	7749	
{5286}						
(9 rows)						

```
-- Сеанс 1
COMMIT
UPDATE 1

-- Сеанс 2
COMMIT
UPDATE 1

-- Сеанс 3
COMMIT
```

Задача 2:

```
CREATE TABLE
INSERT 0 3

-- Сеанс 1
BEGIN
UPDATE 1

-- Сеанс 2
BEGIN
UPDATE 1

-- Сеанс 3
BEGIN
UPDATE 1

-- Сеанс 1
UPDATE 1
-- Ожидание...

-- Сеанс 2
UPDATE 1
-- Ожидание...

-- Сеанс 3
UPDATE 1
ERROR:  deadlock detected
DETAIL:  Process 1568 waits for ShareLock on transaction 301036; blocked by
process 1445.
Process 1445 waits for ShareLock on transaction 301037; blocked by process 1524.
Process 1524 waits for ShareLock on transaction 301038; blocked by process 1568.
HINT:  See server log for query details.
CONTEXT:  while updating tuple (0,1) in relation "deadlock3_test"

-- Логи
2025-11-16 16:12:35.063 MSK [1568] postgres@lab06_db ERROR:  deadlock detected
2025-11-16 16:12:35.063 MSK [1568] postgres@lab06_db DETAIL:  Process 1568 waits
for ShareLock on transaction 301036; blocked by process 1445.
    Process 1445 waits for ShareLock on transaction 301037; blocked by process
1524.
    Process 1524 waits for ShareLock on transaction 301038; blocked by process
1568.
    Process 1568: UPDATE deadlock3_test SET value = 334 WHERE id = 1;
    Process 1445: UPDATE deadlock3_test SET value = 112 WHERE id = 2;
    Process 1524: UPDATE deadlock3_test SET value = 223 WHERE id = 3;
2025-11-16 16:12:35.063 MSK [1568] postgres@lab06_db HINT:  See server log for
query details.
2025-11-16 16:12:35.063 MSK [1568] postgres@lab06_db CONTEXT:  while updating
tuple (0,1) in relation "deadlock3_test"
2025-11-16 16:12:35.063 MSK [1568] postgres@lab06_db STATEMENT:  UPDATE
deadlock3_test SET value = 334 WHERE id = 1;
```

Задача 3:

```
CREATE TABLE
INSERT 0 2

-- Сеанс 1
BEGIN
UPDATE 1

-- Сеанс 2
BEGIN
UPDATE 1

-- Сеанс 1
COMMIT

-- Сеанс 2
UPDATE 1

-- Сеанс 2
COMMIT
```

Часть 4. Блокировки в оперативной памяти**Выполненные задачи:**

1. Используя `pg_buffercache`, убедилась, что открытый курсор удерживает закрепление буфера (pinning_backends > 0) для быстрого чтения следующей строки.
2. Открыла курсор на таблице и, не закрывая его, выполнила `VACUUM` этой таблицы. `VACUUM` не ожидает освобождения закрепления буфера - он пропускает закрепленные страницы.
3. Повторила эксперимент с `VACUUM FREEZE`. `VACUUM FREEZE` должен обработать все страницы, поэтому он ожидает снятия закрепления буфера.

Команды:**Задача 1:**

```
CREATE EXTENSION IF NOT EXISTS pg_buffercache;

CREATE TABLE cursor_test(id INT, data TEXT);
INSERT INTO cursor_test SELECT g, 'data_' || g FROM generate_series(1, 10000) g;
```

```
-- Сеанс 1
BEGIN;
DECLARE cur CURSOR FOR SELECT * FROM cursor_test;
```

```
FETCH 1 FROM cur;

-- Сеанс 2
SELECT
    c.relname,
    b.bufferid,
    b.relblocknumber,
    b.isdirty,
    b.usagecount,
    b.pinning_backends
FROM pg_buffercache b
JOIN pg_class c ON b.relfilenode = pg_relation_filenode(c.oid)
WHERE c.relname = 'cursor_test' AND b.pinning_backends > 0;

-- Сеанс 1
CLOSE cur;
COMMIT;
```

Задача 2:

```
-- Сеанс 1
BEGIN;
DECLARE cur CURSOR FOR SELECT * FROM cursor_test;
FETCH 1 FROM cur;

-- Сеанс 2
VACUUM cursor_test;

-- Проверяем, был ли VACUUM выполнен
SELECT relname, last_vacuum
FROM pg_stat_all_tables
WHERE relname = 'cursor_test';

-- Сеанс 1
CLOSE cur;
COMMIT;
```

Задача 3:

```
-- Сеанс 1
BEGIN;
DECLARE cur CURSOR FOR SELECT * FROM cursor_test;
FETCH 1 FROM cur;
```

```
-- Сеанс 2
VACUUM FREEZE cursor_test;
```

```

SELECT
    pid,
    wait_event_type,
    wait_event,
    state,
    query
FROM pg_stat_activity
WHERE query LIKE '%VACUUM%' AND state = 'active';

```

```

-- Сеанс 1
CLOSE cur;
COMMIT;

```

Фрагменты вывода:

Задача 1

```

CREATE EXTENSION
CREATE TABLE
INSERT 0 10000

-- Сеанс 1
BEGIN
DECLARE CURSOR
    id | data
----+-----
    1 | data_1
(1 row)

-- Сеанс 2
    relname   | bufferid | relblocknumber | isdirty | usagecount | pinning_backends
-----+-----+-----+-----+-----+-----
cursor_test |      400 |              0 | f       |          5 |                1
(1 row)

-- Сеанс 1
CLOSE CURSOR
COMMIT

```

Задача 2:

```

-- Сеанс 1
BEGIN
DECLARE CURSOR
    id | data
----+-----

```



```

1 | data_1
(1 row)

-- Сеанс 2
VACUUM

      relname      |          last_vacuum
-----+-----
cursor_test | 2025-11-16 16:25:04.528867+03
(1 row)

-- Сеанс 1
CLOSE CURSOR
COMMIT

```

Задача 3:

```

-- Сеанс 1
BEGIN
DECLARE CURSOR
id | data
----+-----
1 | data_1
(1 row)

-- Сеанс 2
VACUUM FREEZE cursor_test;
-- Ожидание...

-- Сеанс 3
pid | wait_event_type | wait_event | state | query
-----+-----+-----+-----+-----
-
17234 | BufferPin        | BufferPin   | active | VACUUM FREEZE cursor_test;
(1 row)

-- Сеанс 1
CLOSE CURSOR
COMMIT

-- Сеанс 2
VACUUM

```

Результаты выполнения

1. Мониторинг статистики таблиц

После вставки 5 строк счетчики показали `n_tup_ins = 5`, `n_live_tup = 5`. После удаления: `n_tup_del = 5`, `n_live_tup = 0`, `n_dead_tup = 5` - строки остались как мертвые кортежи. VACUUM очистил мертвые кортежи (`n_dead_tup = 0`) и обновил метку `last_vacuum`.

2. Взаимоблокировки (deadlock)

PostgreSQL автоматически обнаружил deadlock через ~1 секунду. В логах зафиксированы процессы-участники, типы блокировок и граф ожиданий. Одна транзакция получила ошибку "deadlock detected", вторая продолжила выполнение.

3. Расширение pg_stat_statements

Собрана статистика по запросам: количество вызовов, время выполнения, обработанные строки. UPDATE показал 2.543 мс, SELECT - 0.234 мс. Параметризация запросов позволяет группировать похожие запросы для анализа.

4. Блокировки при чтении

При чтении строки на Read Committed захватывается **AccessShareLock** на таблицу. Эта блокировка совместима с SELECT/INSERT/UPDATE, но конфликтует с DDL-операциями. Освобождается при COMMIT/ROLLBACK.

5. Предикатные блокировки

На уровне Serializable при чтении диапазона по индексу фиксируются предикаты. Если транзакции читают пересекающиеся диапазоны и вставляют данные, возникает ошибка сериализации - это ложная ошибка, так как фактического конфликта может не быть.

6. Логирование ожиданий

После настройки **log_lock_waits = on** в логах появляются записи "process X still waiting for ShareLock" с PID процессов, типом блокировки и временем ожидания. Критично для диагностики производительности.

7. Очередь блокировок строк

При обновлении одной строки тремя транзакциями формируется очередь. Первая захватывает блокировку, вторая и третья ждут. В **pg_locks** видна цепочка через **pg_blocking_pids()**. Блокировка передается последовательно после каждого COMMIT.

8. Deadlock трех транзакций

Циклическая зависимость $T1 \rightarrow T2 \rightarrow T3 \rightarrow T1$ создает deadlock. PostgreSQL прерывает одну транзакцию. Логи содержат полный граф зависимостей, что позволяет понять причину и предотвратить повторение.

9. Deadlock с одним UPDATE невозможен

Две транзакции с одним UPDATE не создают deadlock - вторая просто ждет первую, но не захватывает нужную первой блокировку. Нет обратной зависимости → нет deadlock.

10. Закрепление буферов

Открытый курсор закрепляет буфер (**pinning_backends = 1**) для быстрого FETCH. Обычный VACUUM и VACUUM FREEZE пропускают закрепленные страницы. Ожидание возникает только при VACUUM FULL с эксклюзивной блокировкой.

Выводы

В ходе данной лабораторной работы изучила систему блокировок в PostgreSQL и методы мониторинга активности сервера. Получила практические навыки анализа статистики, диагностики блокировок и взаимоблокировок, а также использования инструментов мониторинга.